

高德地图AJX容器工具链从0到1建设

AJX容器工具链发展历程

石器时代：使用JavaScript通用编辑器编写代码，无API提示及补全、无语法高亮，只能print打印log调试；

现代化：AJX IDE集成开发调试环境，为AJX量身定制，满足各种开发调试需求；编译扫码、代码高亮、API提示及补全、断点调试、性能排查等；

石器时代

代码编写：

无API提示及补全、无语法高亮

```
ajax.addEventListener('ready', function() {
  let xml = `
    <div id='layout' style='margin-top:50px;'>
      <div id='body' style='height:101.4px;width:91px;align-self:center;justify-content:center;position:relative;'>
        <img syncload='true' src='xxx.png' style='width:91px;height:101.4px;align-self:center;' />
        <img syncload='true' src='test.png' />
      </div>
    </div>
  `;

  let css = `
    .demo-page {
      flex-grow: 1;
    }
    .demo-page_title {
      color: #00f;
      text-align: center;
      padding: 20px 0;
      width: 100%;
      background-color: #f8f8f8;
    }
  `;

  let node = ajax.ui.inflate(xml);
  ajax.ui.applyStyle(css);
  ajax.ui.root.appendChild(node);
});
```

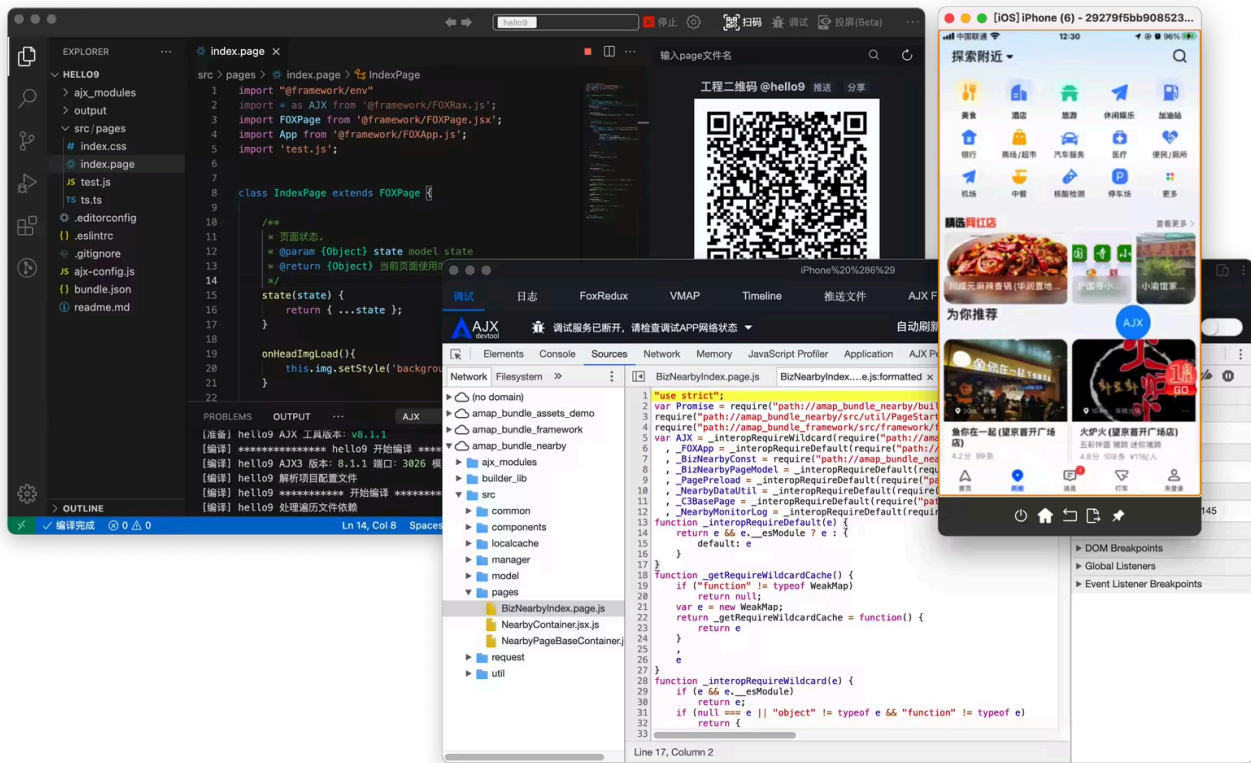
调试：

只能日志输出，无法断点调试；

```
print("hello AJX!!!");
```

现代化

现代化的开发工具体系，提供了AJX语法高亮、智能API提示，语法校验、编译扫码、断点调试等一些列高效开发工具；下面我们看看各种工具在AJX上的实现及相对于传统web工具上的差异；



AJX代码编写

AJX容器在架构设计上参考了web容器实现，在业务代码编写上选择了JavaScript，UI描述使用XML、CSS，DOM接口设计上也参考了W3C的一系列标准，大部分接口设计都是与W3C对齐的。差别比较大的地方在BOM接口层，这部分需要对接大量的native能力与传统web差别较大。

由此可看代码编写可以用传统web开发类IDE，如vscode、webstorm、sublime等；

AJX容器推荐使用的IDE为vscode(免费开源易扩展默认就支持了大量web开发相关的语言特性)。由于AJX部分接口设计与web的差异性，在web开发环境中一些AJX特有的接口在语法高亮、自动补全、语法检查等特性上就会缺失。

语法高亮

•AJX API语法的高亮提示，基于vscode Language Features实现

```
componentDidMount() {  
  const osName = ajax.os.name;  
  ajax.addEventListener('ready', () => {  
    print(osName);  
    ajax.ui.applyStyle(this.cssText);  
  })  
}
```

- AJX CSS语法高亮，扩展CSS TextMate语法配置，增加AJX CSS特定语法

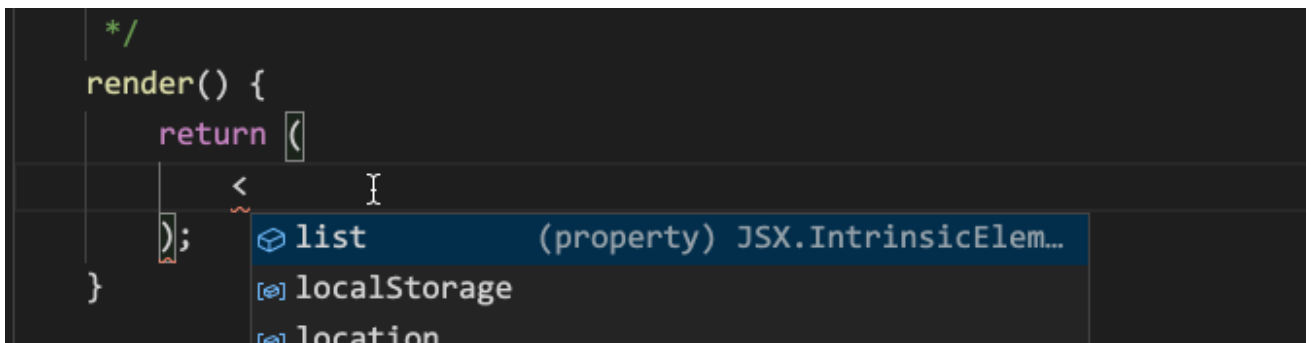
```
.demo {  
  imagestretch: 50 50 50 50;  
  hoverboundary:true;  
  fill-mode: contain;  
}
```

- AJX XML语法高亮，扩展HTML TextMate语法配置，增加AJX XML特定语法

```
return (  
  <div class="container">  
    <richtext text="Hello World"></richtext>  
  </div>  
)
```

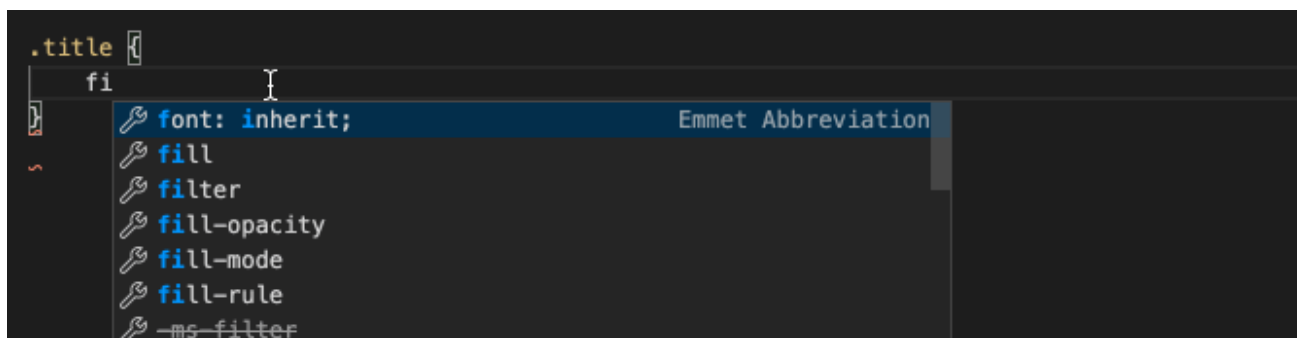
自动补全

- XML补全，通过vscode Language Extensions API registerCompletionItemProvider，来实现AJX XML组件名及属性相关功能自动补全能力

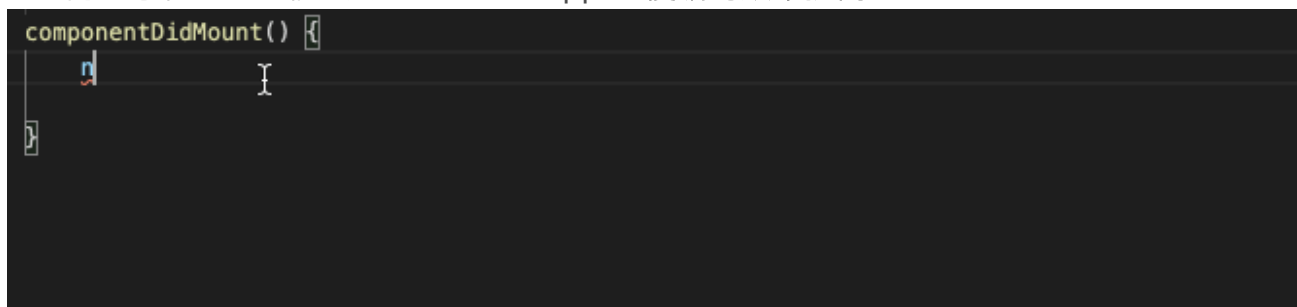


- CSS补全，扩展vscode内置扩展css-language-features能力，增加AJX CSS描述信息，修改JS中CSS驼峰补全方式(AJX不支持)；



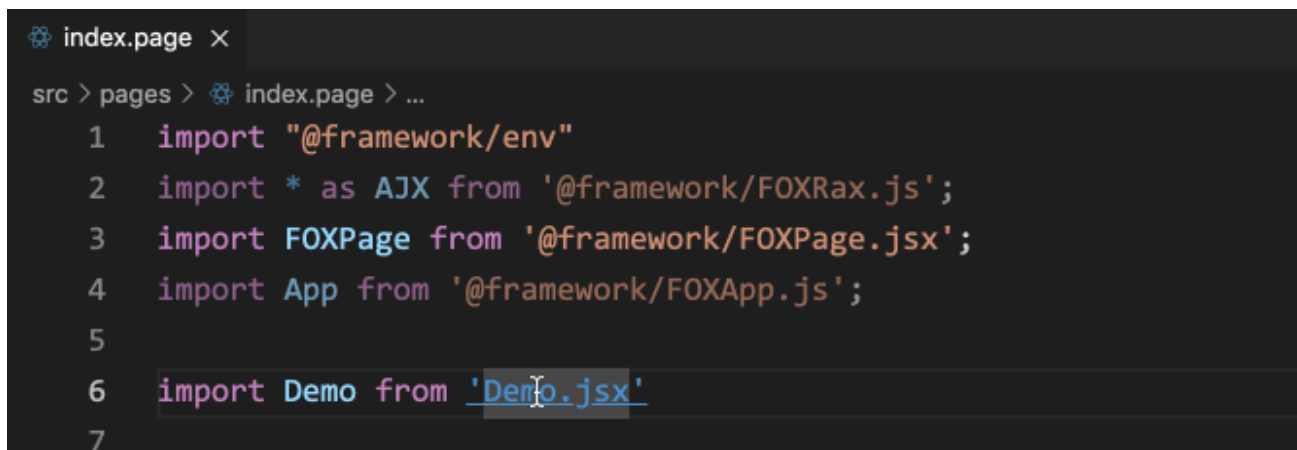


- AJX API补全，扩展typescript server增加AJX API d.ts文件的读取识别能力，实现AJX API的自动补全；同时对常用API我们定义了一些列的snippets使编码效率提高；



代码导航

由于AJX有一套自己的包管理模式，代码中依赖模块的导入方式与传统web不一致，AJX是全局唯一文件名方式引用，导致vscode默认代码导航是无法找到对应依赖文件路径的，因此需要扩展typescript的依赖文件路径解析能力实现AJX的代码导航；



AJX编译

编译就是把业务编写的代码，转换成AJX engine能够识别的目标代码的过程，相对于传统web编译，我们AJX在编译上有哪些区别：

- AJX有自己的一套包管理系统(AJX包统称bundle)，与传统web开发npm包管理有所区别，AJX目前不支持安装npm模块，依赖配置方式及bundle配置信息也有所区别；

```
{ } bundle.json > ...  
1  {  
2    "name": "demo",  
3    "bundleType": "business",  
4    "dependencies": {  
5      "framework": "1.8.0"    依赖配置  
6    }  
7  }
```

▼ AMAP_BUNDLE_DRIVE

▼ ajax_modules

依赖模块管理

- > amap_bundle_autotrack
- > amap_bundle_emojisource
- > amap_bundle_fcajaxcloud
- > amap_bundle_lib
- > amap_bundle_lib_app
- > amap_bundle_lib_aux
- > amap_bundle_lib_basemap
- > amap_bundle_lib_card_service
- > amap_bundle_lib_content_value add

- 在模块及文件引用路径编写定义上也有区别，AJX支持当前模块全局唯一文件名查找

```

8  import {getHasShowTips} from 'TripSearchLocalStorageUtil.js'; 当前模块唯一
9  import TripSearchSuperIdLogic from 'TripSearchSuperIdLogic.js';
10 import TripSearchAlcLog from 'TripSearchAlcLog.js';
11 import PoiFullUtLogLogic from 'PoiFullUtLogLogic.js';
12 import DriveLogUtil from '@amap_bundle_lib_drivecommon/DriveLogUtil.js';
13 import { savePoiToHistory } from 'sugUtil.js';
14 import CRPUtLogConstUtil from '@amap_bundle_lib_drive/CRPUtLogConstUtil.js';
15 import TripConstUtil from '@amap_bundle_lib_drivecommon/VehicleConstUtil.js';
16
17  /**
    外部模块文件依赖写法
  
```

- 图片资源处理方式不一样，native环境有2x、3x类图片，云化图片资源，预加载云化资源等这些都需要在编译时处理；

```

<img
  src={require.toUrl('@framework/race-panel-bg.web3.png')}
  class="arrive_card_bg_img" />
<div class="content_section_top">
  <div
    class="back_icon_container"
    onClick={this.handleBack}>
    <svg
      class="back_icon"
      fillcolor="@Color_Text_Highlight"
      src={require.toUrl('@framework/ic-arrow-left-o.svg')} />
    </div>
    <img
      src={isArrived ? require.toUrl('arrive_card_title.png') : require.to
      class={cardTitleCls} />
    </div>
  
```

云化图片资源

2x、3x资源图片

```

12  excludes: {
13    dev: ['scripts/**'],
14    release: ['scripts/**', '**/TestGuidePage.page', '**/test_guide_page.css']
15  },
16  preloadWeb: [
17    'taxi_multi_cp_poster.web.png',
18    'wait-redpack-popup-0-cp.web.png',
19    'wait-redpack-popup-1-cp.web.png',
20    'wait-redpack-popup-2-cp.web.png',
21    'personal_center_bac.web.png'
22  ]
23 };
  
```

预加载云化资源配置

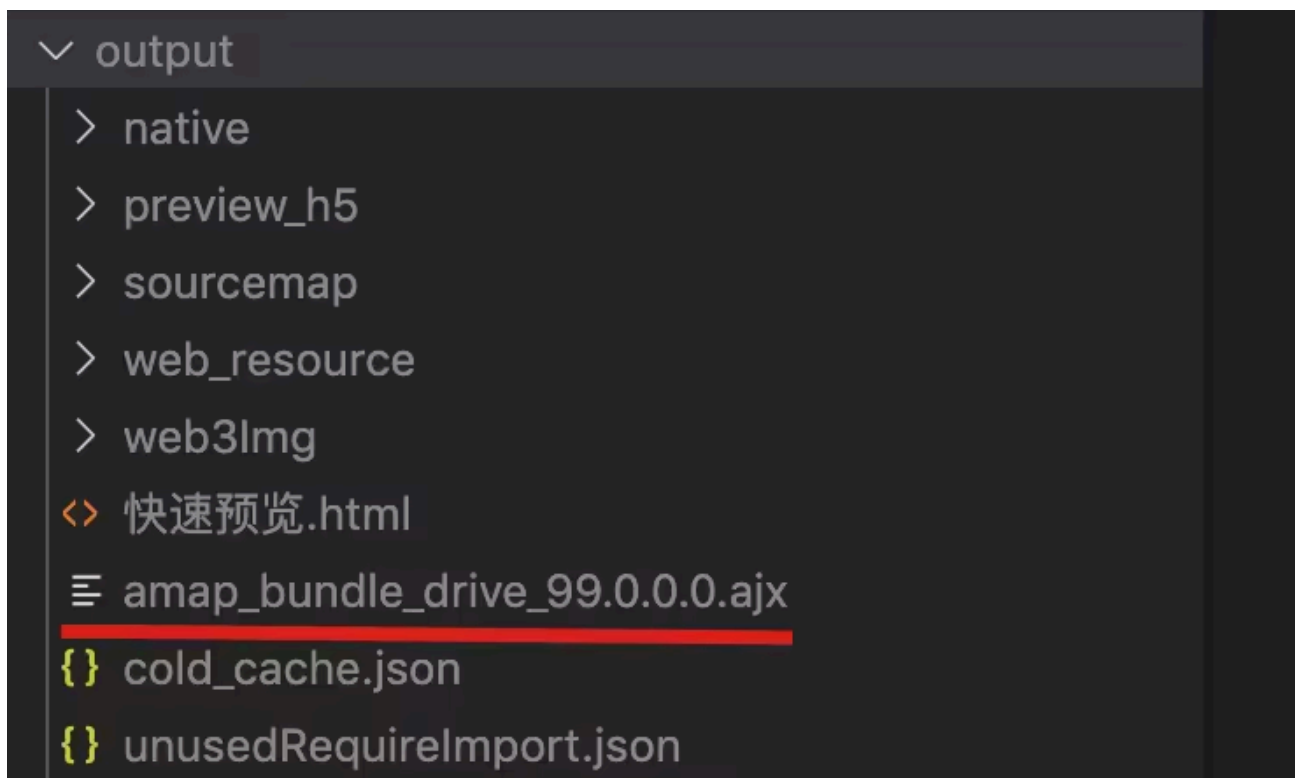
- 页面配置信息编译，每个AJX页面可以有一个

`page.json` 配置文件，用来描述页面UI渲染信息、地图配置信息等一些列需要在页面渲染前AJX engine需要知道的信息，因为是json格式描述的为了动态化，支持模板语法编写，配置信息里面的路径信息，这些都需要编译处理；

```
1  {
2      "showMap": true,
3      "dsl": {
4          "vmap": {
5              "localSettings":["modeState"],
6              "noDiffButRecover":["modeState"],
7              "mapState": {
8                  "modeState": {
9                      "nMode": 0,
10                     "nTime": 0,
11                     "nState": 1,
12                     "nSimple3DEnable": 0
13                 }
14             }
15         },
```

- 打包输出，传统web打包方式是把所有依赖文件，打到一个文件里面去最终产物为一个文件，AJX打包产物为

`.ajx` 文件，AJX编译是把多个文件压缩到一个 `.ajx` 文件中，`.ajx` 有一些列文件压缩和读取算法以减小包大小和提升文件读取速度，`.ajx` diff算法可以基于另外一个 `.ajx` 生成diff `.ajx` 文件以减小热更新的包体积大小；



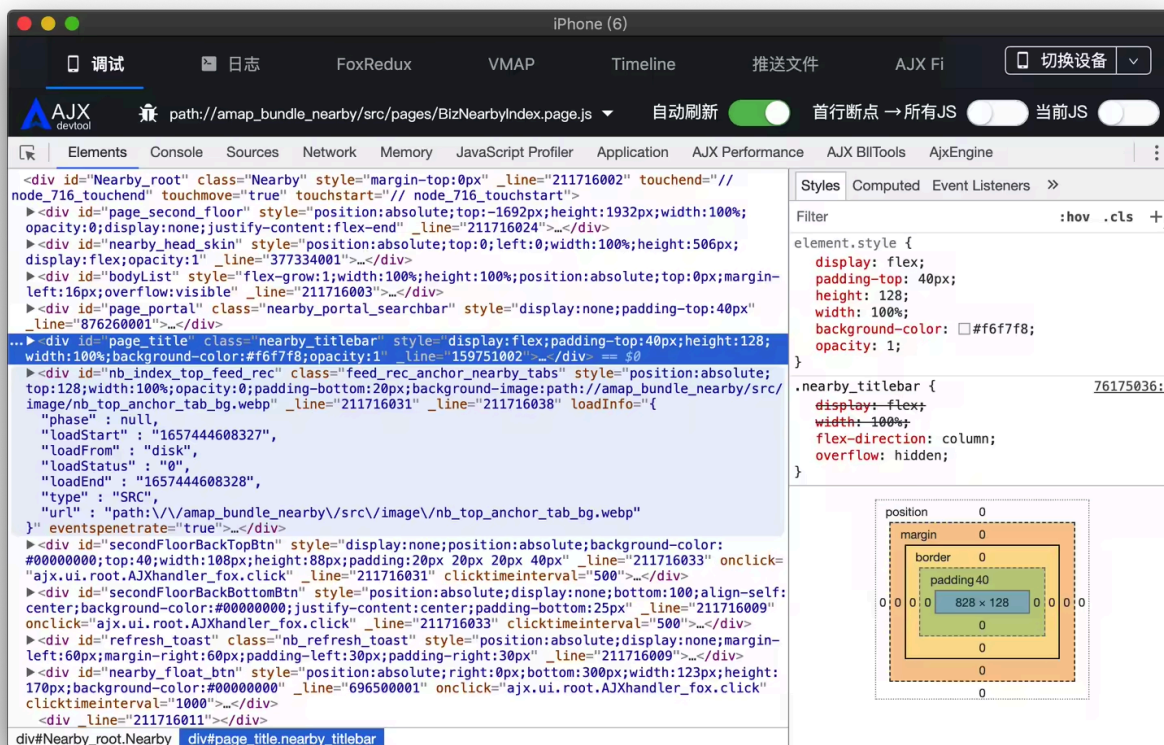
AJX调试

AJX容器无法直接使用web调试工具来调试，是因为它们的运行环境不同，AJX代码的运行环境为native环境；它们相同的地方在于都使用了javascript引擎来运行业务代码，常用的JS调试工具为chrome devtools，它不光可以调试网页中的JS代码，还可以调试node中的js代码；那AJX代码是不是也可以调试呢，当然可以，chrome devtools本身就是开源的，它本身就是基于web技术开发的一套调试相关的网页应用运行在浏览器环境，它跟调试的目标代码运行环境没有关系，只需要目标环境对接chrome devtools的调试协议就能实现对应环境的调试能力了。AJX容器的调试能力就是基于chrome devtools建设的，调试体验大体上与web一致，web调试常用的JS、XML、CSS、Network、Console等调试能力都支持，同时我们还扩展了原有的调试协议，支持了AJX相关的一些调试能力如日志、vmap、timeline、沙箱等；由于调试工具不是集成在运行环境中属于remote debug，需要手机与调试工具建立链接，为方便调试链接建立，我们提供了扫码连接、USB直连等连接方式；

•扫码调试

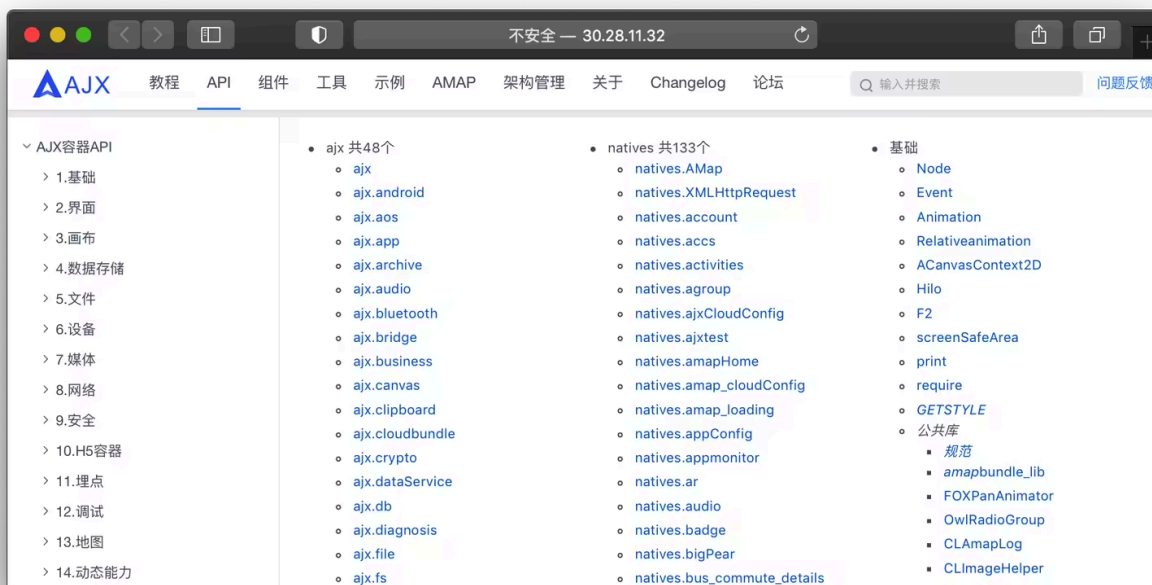


•AJX Devtools



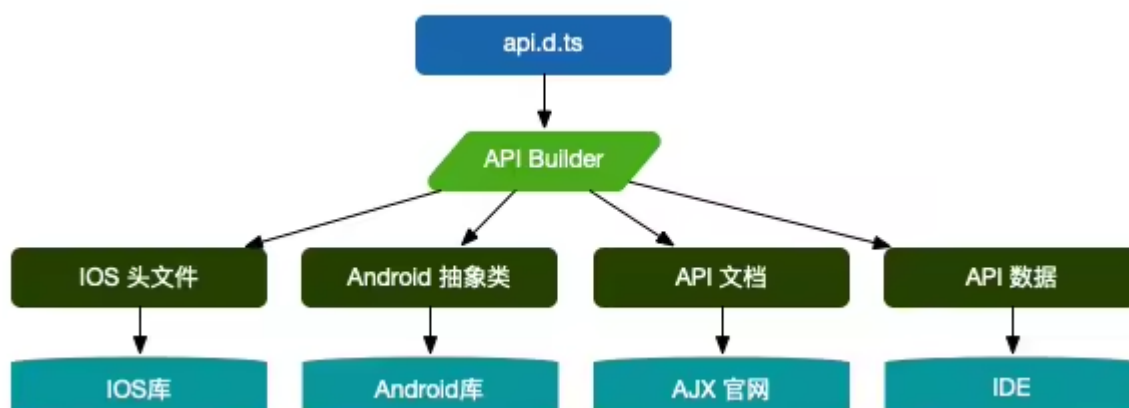
AJX文档

一个好的跨端容器，不光能完美的解决各种跨端问题，还要能提供良好的开发体验，良好的开发体验必须要有完善的文档体系，AJX官网提供了AJX教程、API文档、示例demo、在线试一试等一系列功能，提供了全文搜索能力方便文档的快速查找，文档支持评论及点赞方便文档负责人对文档质量的把控，文档维护人方便研发在看文档遇到问题能及时找到对应的接口人及时解决问题；



怎么保证API文档的及时更新，所有API通过typescript定义描述，通过API Builder(API解析生成工具)生成文档、iOS头文件及Android抽象类，新生成文档会自动同步到官网；iOS头文件及Android抽象类用来约束native API实现规范；

API数据同时会同步给IDE，IDE会及时更新API自动补全相关的数据保证提示信息跟官网是一致的。



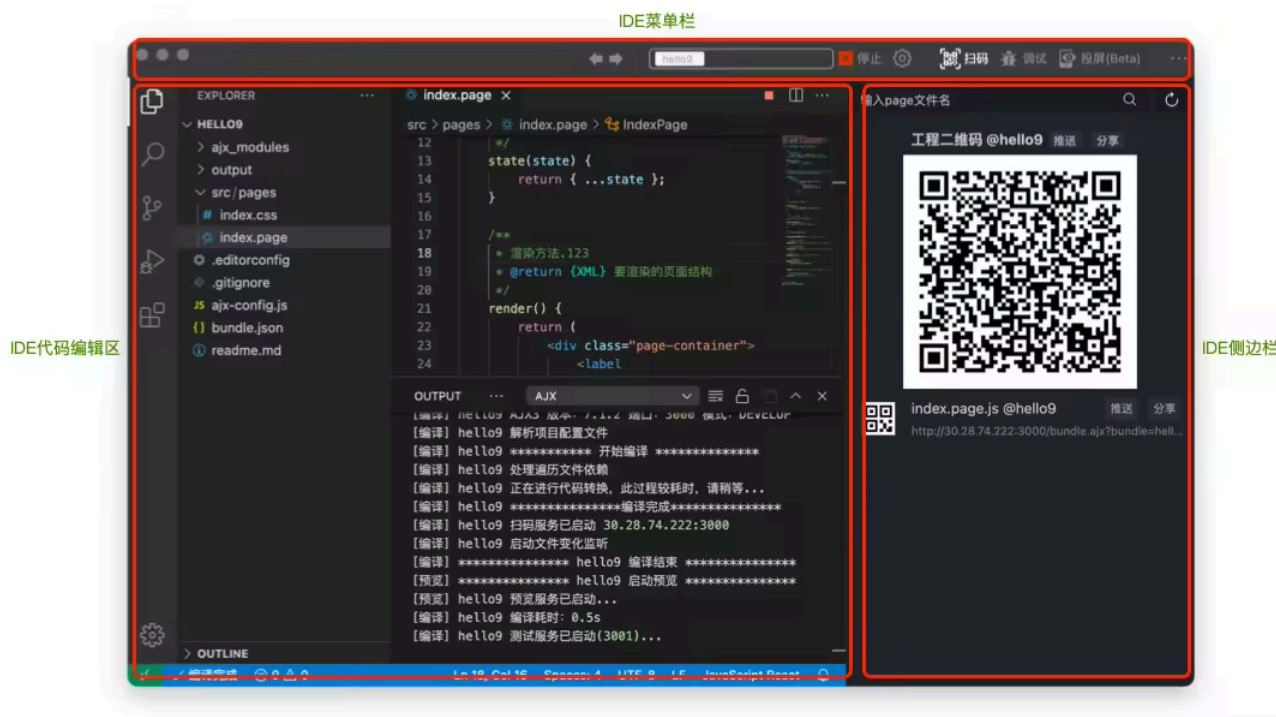
AJX IDE

为啥AJX需要自己的IDE?

AJX推荐使用的IDE为vscode，vscode是一个轻量且功能强大的源代码编辑器，提供了强大的扩展能力，可以通过扩展能力实现很多自定制化需求；AJX开发工具是否可以通过vscode的扩展能力来实现，答案是可以的，但是研发体验会比较差；因为vscode提供的扩展能力有很大的局限性，特别是UI交互上的自定制化能力很弱；导致通过扩展实现的很多功能跟IDE主功

能交互是隔离的，而且需要使用大量命令行调用方式，研发使用成本也会大幅提高；我们要的不光是集成式开发环境，还要开发体验的流畅一致；因此我们基于vscode开源项目扩展实现了AJX IDE。

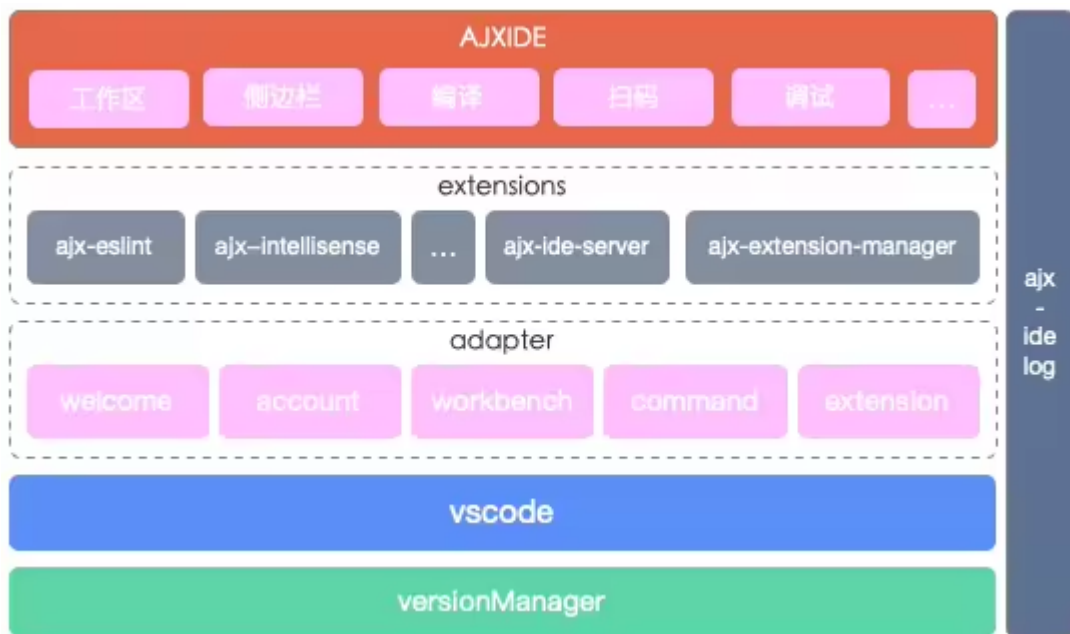
•AJX IDE窗口布局



•AJX IDE功能设计



•AJX IDE架构设计



AJX工具链未来规划

- 优化部分工具使用体验，有些工具实现的比较仓促，有功能不完善及交互体验差的问题，需要不断迭代升级；
- 增强各工具间关联性，达到工具链的效果，在各个调试环节都能互联互通；
- 补全调试环节中缺失的部分；
- 业务自定义调试能力支持；
- 文档能力增强，官网重构支持更多AJX特色功能；

更多AJX工具建设正在进行中，欢迎大家交流拍砖。